Лабораторная работа №2.

Управление Docker-контейнерами

Тема: Запуск Nginx, публикация портов, команды docker ps, stop, start, restart, rm, exec и logs

Цель: Научиться запускать, проверять, останавливать, перезапускать и удалять контейнеры; выполнять команды внутри Nginx и проверять веб-сервер.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Продолжительность: 4 академических часа (180 минут). Работа выполняется в Debian 13. Docker Engine уже установлен в лабораторной №1. Все команды Docker выполняются через sudo. Установку Docker повторять не нужно.

2. Цель и результаты

Цель: научиться управлять контейнерами на примере готового образа Nginx.

1. Отличать образ от контейнера.
2. Запускать веб-сервер в фоновом режиме.
3. Проверять HTTP-доступ через localhost.
4. Использовать команды ps, stop, start, restart, rm.
5. Выполнять простые команды внутри контейнера.
6. Просматривать логи и запускать два независимых контейнера.

3. Необходимое оборудование

- Debian 13 с установленным Docker Engine.
- Права sudo и доступ в интернет для загрузки образа Nginx.
- Терминал Debian; curl установлен либо доступен браузер в Debian.

4. Подготовка

Откройте терминал Debian. Убедитесь, что служба Docker работает.

```
Проверяем Docker

sudo systemctl status docker
```

Ожидается Active: active (running). Нажмите q для выхода. Если служба выключена, запустите её:

```
При необходимости запускаем службу

sudo systemctl start docker
```

```
Посмотрите существующие контейнеры

sudo docker ps -a
```

Если контейнеры web1 или web2 уже существуют после прежних упражнений, используйте отдельную учебную виртуальную машину либо согласуйте удаление этих тестовых контейнеров с преподавателем. Не удаляйте чужие контейнеры.

5. Задание 1. Загрузка образа

```
Загрузите готовый образ Nginx
```

```
sudo docker pull nginx:stable
```

```
Проверьте список образов
```

```
sudo docker images
```

Найдите строку nginx со значением stable в столбце TAG. Запишите, чем образ отличается от контейнера.

6. Задание 2. Запуск веб-сервера

```
Создайте и запустите Nginx
```

```
sudo docker run -d --name web1 -p 127.0.0.1:8080:80 nginx:stable
```

Пояснение: -d — работа в фоне; --name web1 — имя контейнера; 127.0.0.1:8080:80 — обращения к порту 8080 самой Debian перенаправляются на порт 80 внутри контейнера. Этот вариант не публикует страницу для других компьютеров.

```
Проверьте, что контейнер работает
```

```
sudo docker ps
```

Найдите имя web1 и состояние Up.

```
Проверьте веб-страницу
```

```
curl http://localhost:8080
```

Ожидается HTML-код страницы Welcome to nginx! Если команда curl не найдена, установите её через sudo apt install curl или проверьте страницу в браузере Debian.

7. Задание 3. Просмотр журналов

```
Посмотрите последние 10 строк журнала
```

```
sudo docker logs --tail 10 web1
```

Найдите запись о запросе к странице. Если записей нет, повторите curl http://localhost:8080 и снова откройте журнал.

8. Задание 4. Работа внутри контейнера

Откройте командную оболочку контейнера

```
sudo docker exec -it web1 sh
```

Приглашение терминала изменится: теперь команды выполняются внутри контейнера.

Выполните внутри контейнера

```
pwd
ls
exit
```

pwd показывает каталог, ls — файлы, exit возвращает в Debian. После выхода выполните sudo docker ps и убедитесь, что контейнер всё ещё работает.

9. Задание 5. Остановка и повторный запуск

Остановите контейнер

```
sudo docker stop web1
```

Посмотрите все контейнеры

```
sudo docker ps -a
```

Для web1 ожидается состояние Exited. Проверка curl http://localhost:8080 должна завершиться ошибкой подключения.

Запустите существующий контейнер

```
sudo docker start web1
```

Проверьте работу страницы

```
curl http://localhost:8080
```

Объясните в отчёте, почему использовали start, а не run.

10. Задание 6. Перезапуск

Перезапустите контейнер

```
sudo docker restart web1
```

Убедитесь, что он работает

```
sudo docker ps
```

```

```

Имя web1 должно сохраниться. Зафиксируйте состояние в отчёте.

11. Задание 7. Второй независимый Nginx

```
Создайте второй контейнер
```

```
sudo docker run -d --name web2 -p 127.0.0.1:8081:80 nginx:stable
```

```
Проверьте оба адреса
```

```
curl http://localhost:8080
curl http://localhost:8081
```

```
Посмотрите оба контейнера
```

```
sudo docker ps
```

Оба используют один образ, но разные имена и разные порты Debian. Объясните, почему оба могут использовать порт 80 внутри своих контейнеров.

12. Задание 8. Удаление учебных контейнеров

```
Остановите и удалите контейнеры
```

```
sudo docker stop web1 web2
sudo docker rm web1 web2
```

```
Проверьте результат
```

```
sudo docker ps -a
sudo docker images
```

web1 и web2 должны исчезнуть из списка контейнеров. Образ nginx:stable остаётся локально.

13. Индивидуальное задание

7. Создайте контейнер student-web из nginx:stable.
8. Назначьте ему порт Debian 8090, ограниченный адресом 127.0.0.1.
9. Проверьте ответ http://localhost:8090.
10. Получите последние 5 строк его журнала.
11. Остановите контейнер, покажите его в списке всех контейнеров.
12. Запустите снова, проверьте страницу.
13. Остановите и удалите контейнер, но не удаляйте образ.

Команды для индивидуального задания составьте самостоятельно по образцу предыдущих этапов.

14. Что включить в отчёт

14. ФИО, группа, тема и цель.
15. Результат docker images и docker ps.
16. Команда запуска web1 и пояснение параметров.
17. Скриншот страницы или результат curl.
18. Сведения из docker logs.
19. Результат docker exec (pwd, ls).
20. Результаты stop, start и restart.
21. Подтверждение работы web1 и web2.
22. Результат удаления и сохранения образа.
23. Результаты индивидуального задания и вывод.

15. Контрольные вопросы

24. Чем Docker Image отличается от Docker Container?
25. Зачем нужен параметр -d?
26. Для чего нужен --name?
27. Что означают числа 8080:80?
28. Чем docker ps отличается от docker ps -a?
29. Чем docker start отличается от docker run?
30. Что делает docker restart?
31. Зачем используется docker exec?
32. Почему после exit из sh Nginx продолжает работать?
33. Что можно узнать через docker logs?
34. Почему два контейнера могут иметь порт 80 внутри?
35. Почему после docker rm образ nginx:stable остаётся?

16. Оценивание

Критерии оценивания

Критерий	Баллы
Подготовка и загрузка образа	10
Запуск Nginx и HTTP-проверка	20
Журналы и docker exec	15
stop, start, restart	20
Два контейнера	15
Индивидуальное задание	10
Отчёт и ответы	10
Итого	100

Рекомендуемое распределение времени: подготовка — 15 минут; Nginx — 35; журналы и exec — 30; жизненный цикл — 35; второй контейнер — 20; индивидуальное задание — 25; отчёт и защита — 20 минут.